

Phase 1: Selection and Filteration

This notebook demonstrates fundamental data operations:

- **Selection:** Choosing specific columns from datasets
- **Filteration:** Applying conditions to filter rows

Objectives:

1. Load customer data
2. Select relevant columns
3. Apply filtering conditions
4. Analyze filtered results

```
In [ ]: # Load the customer report data
        from pyspark.sql import functions as F

        # Read customer data
        df_customers = spark.table("workspace.default.customer_report")

        print(f"Total records loaded: {df_customers.count():,}")
        print(f"\nSchema:")
        df_customers.printSchema()
```

Total records loaded: 454,717

Schema:

root

```
|-- customer_name: string (nullable = true)
|-- city: string (nullable = true)
|-- total_spend: decimal(29,2) (nullable = true)
|-- order_count: long (nullable = true)
|-- segment: string (nullable = true)
|-- first_order_date: date (nullable = true)
|-- last_order_date: date (nullable = true)
```

```
In [ ]: # Display sample data
print("Sample of customer data:")
display(df_customers.limit(10))
```

Sample of customer data:

customer_name	city	to
Customer#000721180	xVvi2xMYV7MqiPdC	71
Customer#000382414	Qjl aHILRIBLeg6HkUjg	71
Customer#000299701	bQayuljeavArxOQl	71
Customer#000179275	t6afpRmxArdPPqqQPSzVRWfpyTxECkQsWaTX	68
Customer#000292987	CjD	68
Customer#000253099	fDBY7k7eegbOws6DnsEEyr	68
Customer#000333100	OB	67
Customer#000370441	eYacGNQvE2lmiVMM7zK5vHhGwz3wEl8EXQa	67

```
In [ ]: # SELECTION: Choose specific columns
selected_columns = df_customers.select(
    "customer_name",
    "city",
    "segment",
    "total_spend",
    "order_count"
)

print("Selected columns: customer_name, city, segment, total_spend,
order_count")
display(selected_columns.limit(10))
```

Selected columns: customer_name, city, segment, total_spend,
order_count

customer_name	city	segment
Customer#000721180	xVvi2xMYV7MqiPdC	Gold
Customer#000382414	Qjl aHILRIBLeg6HkUjg	Gold
Customer#000299701	bQayuljeavArxOQl	Gold
Customer#000179275	t6afpRmxArdPPqqQPSzVRWfpyTxECkQsWaTX	Gold
Customer#000292987	CjD	Gold
Customer#000253099	fDBY7k7eegbOws6DnsEEyr	Gold
Customer#000333100	OB	Gold
Customer#000370441	rYocGNGvE2lmiVMV7zKExlHsGw3zxEk8EYQs	Gold

```
In [ ]: # FILTERATION: Apply conditions to filter rows

# Filter 1: Gold segment customers only
gold_customers = df_customers.filter(F.col("segment") == "Gold")

print(f"Gold segment customers: {gold_customers.count():,}")
print("\nSample of Gold customers:")
display(gold_customers.limit(10))
```

Gold segment customers: 454,716

Sample of Gold customers:

customer_name	city	to
Customer#000721180	xVvi2xMYV7MqiPdC	71
Customer#000382414	Qjl aHILRIBLeg6HkUjg	71
Customer#000299701	bQayuljeavArxOQl	71
Customer#000179275	t6afpRmxArdPPqqQPSzVRWfpyTxECkQsWaTX	68
Customer#000292987	CjD	68
Customer#000253099	fDBY7k7eegbOws6DnsEEyr	68
Customer#000333100	OB	67
Customer#000270441	xYacGNQvE2lmiVMV7zK5vH6Gw37vE18FYQa	67

```
In [ ]: # Filter 2: High-value customers (spend > $5M)
high_value_customers = df_customers.filter(F.col("total_spend") >
5000000)

print(f"High-value customers (>$5M): {high_value_customers.count():,}")
print("\nTop high-value customers:")
display(high_value_customers.orderBy(F.desc("total_spend")).limit(15))
```

High-value customers (>\$5M): 2,366

Top high-value customers:

customer_name	city	to
Customer#000721180	xVvi2xMYV7MqiPdC	71
Customer#000382414	Qjl aHILRIBLeg6HkUjg	71
Customer#000299701	bQayuljeavArxOQl	71
Customer#000179275	t6afpRmxArdPPqqQPSzVRWfpyTxECkQsWaTX	68
Customer#000292987	CjD	68
Customer#000253099	fDBY7k7eegbOws6DnsEEyr	68
Customer#000333100	OB	67
Customer#000370441	xYocGNGvE2lmiVMV7zKExlHsGw2zxEk8EYQs	67

```
In [ ]: # Filter 3: Combined conditions - Gold segment AND high-value
gold_high_value = df_customers.filter(
    (F.col("segment") == "Gold") &
    (F.col("total_spend") > 5000000)
)

print(f"Gold high-value customers: {gold_high_value.count():,}")
print("\nDistribution by city:")
city_distribution =
gold_high_value.groupBy("city").count().orderBy(F.desc("count"))
display(city_distribution.limit(10))
```

Gold high-value customers: 2,366

Distribution by city:

city	count
	39
u	2
S	2
F	2
i	2
0	2
1	2
E	2

```
In [ ]: # Filter 4: Active customers (order_count >= 30)
active_customers = df_customers.filter(F.col("order_count") >= 30)

print(f"Active customers (>=30 orders): {active_customers.count():,}")
print("\nSegment breakdown:")
segment_breakdown = active_customers.groupBy("segment").agg(
    F.count("*").alias("customer_count"),
    F.sum("total_spend").alias("total_revenue"),
    F.avg("total_spend").alias("avg_spend")
)
display(segment_breakdown)
```

Active customers (>=30 orders): 4,958

Segment breakdown:

segment	customer_count	total_revenue	avg_spend
Gold	4958	23594330761.33	4758840.411724

```
In [ ]: # Summary statistics for filtered data
print("=" * 60)
print("PHASE 1 SUMMARY: Selection and Filtration Results")
print("=" * 60)

summary_stats = {
    "Total Customers": df_customers.count(),
    "Gold Customers": gold_customers.count(),
    "High-Value (>$5M)": high_value_customers.count(),
    "Gold + High-Value": gold_high_value.count(),
    "Active (>=30 orders)": active_customers.count()
}

for metric, value in summary_stats.items():
    print(f"{metric:.<50} {value:>10},")

print("=" * 60)
print("Phase 1 Complete: Data selected and filtered successfully")

=====
PHASE 1 SUMMARY: Selection and Filtration Results
=====
Total Customers..... 454,717
Gold Customers..... 454,716
High-Value (>$5M)..... 2,366
Gold + High-Value..... 2,366
Active (>=30 orders)..... 4,958
=====
Phase 1 Complete: Data selected and filtered successfully
```

```
In [ ]:
```

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).